

App Balloo API

Backend API server для мессенджера App Balloo с поддержкой E2E шифрования и Яндекс.интеграций.

Архитектура

База Данных

- **НЕ отдельный PostgreSQL сервер**
- Используется **встроенная SQLite** (better-sqlite3)
- База данных поднимается **на том же процессе**, что и API
- Все данные хранятся локально на сервере
- Автоматическое создание и миграции схем при старте

Безопасность и Шифрование

- **E2E шифрование**: Все сообщения шифруются на клиенте (AES-256-GCM)
- **RSA-2048**: Обмен ключами между пользователями
- **Ключи шифрования НЕ хранятся на сервере**
- Сервер передаёт только зашифрованные данные
- **Файлы шифруются** перед загрузкой на Яндекс.Диск
- **Яндекс токены шифруются** перед сохранением в БД

Яндекс.Интеграции

- **Яндекс.Авторизация**: OAuth 2.0 вход через Яндекс
- **Яндекс.Диск**: Хранение файлов и медиа
- Client ID/Secret хранятся только на сервере в .env
- Автоматическое обновление токенов через refresh_token

Tech Stack

- **Runtime**: Node.js
- **Framework**: Express.js
- **Database**: Better-SQLite3 (embedded, process-local)
- **Authentication**: JWT
- **Real-time**: WebSocket (ws)
- **Encryption**: AES-256-GCM, RSA-2048
- **File Storage**: Яндекс.Диск (через OAuth)
- **Logging**: Winston

Project Structure

```
api/  
├── src/  
│   └── index.js           # Application entry point
```

```

├── config/                # Configuration
│   ├── database.js       # SQLite setup & migrations
│   ├── yandex.js        # Yandex OAuth & Disk config
│   └── encryption.js     # Crypto utilities
├── controllers/          # Route controllers
│   ├── auth.controller.js
│   ├── yandex-auth.controller.js
│   ├── users.controller.js
│   ├── chats.controller.js
│   ├── messages.controller.js
│   ├── files.controller.js
│   └── ...
├── middleware/           # Custom middleware
│   ├── auth.js           # JWT authentication
│   ├── validation.js     # Request validation
│   ├── errorHandler.js  # Error handling
│   └── rateLimiter.js    # Rate limiting
├── routes/               # API routes
│   ├── index.js
│   ├── auth.routes.js
│   ├── yandex-auth.routes.js
│   ├── users.routes.js
│   ├── chats.routes.js
│   ├── messages.routes.js
│   └── ...
├── services/             # Business logic
│   ├── email.service.js
│   ├── yandex-disk.service.js
│   ├── encryption.service.js
│   └── notification.service.js
├── database/             # Database layer
│   ├── schema.js        # Table definitions
│   ├── queries.js       # SQL queries
│   └── migrations.js    # DB migrations
├── websocket/           # WebSocket handler
│   └── index.js
├── tests/                # Test files
├── data/                 # SQLite database files
├── uploads/              # Temporary file uploads
├── logs/                 # Log files
├── .env.example
├── .gitignore
├── package.json
├── README.md
└── TODO.md               # Full API specification

```

Setup

Предварительные требования

- Node.js >= 18

- npm >= 9

Установка

1. Перейти в директорию API:

```
cd api
```

2. Установить зависимости:

```
npm install
```

3. Создать файл `.env` из примера:

```
cp .env.example .env
```

4. Настроить переменные окружения:

```
# Отредактировать .env и заполнить:  
# - JWT_SECRET  
# - YANDEX_CLIENT_ID и YANDEX_CLIENT_SECRET  
# - EMAIL настройки  
# - ENCRYPTION_KEY
```

5. Инициализировать базу данных (создаст SQLite файл):

```
npm run db:init
```

6. Запустить сервер:

```
npm run dev
```

API будет доступен по адресу: <http://localhost:3001>

Документация API

Полное описание всех эндпоинтов находится в [TODO.md](#).

Основные разделы:

1. **Аутентификация** - Регистрация, вход, JWT токены
2. **Яндекс.Авторизация** - OAuth 2.0 через Яндекс
3. **Пользователи** - Профиль, настройки, поиск
4. **Чаты** - Создание, управление, участники
5. **Сообщения** - Отправка, редактирование, E2E шифрование
6. **Файлы и Яндекс.Диск** - Загрузка, хранение, шифрование
7. **Группы** - Роли, права, управление
8. **Приглашения** - Приглашательные ссылки
9. **Уведомления** - Push и внутренние
10. **Администрирование** - Управление пользователями, аналитика
11. **WebSocket** - Реальные события

Безопасность

E2E Шифрование

- Сообщения шифруются **на клиенте** перед отправкой
- Сервер получает только зашифрованные данные
- Ключи шифрования хранятся **только на устройствах пользователя**
- Используется AES-256-GCM для сообщений
- RSA-2048 для обмена ключами

Яндекс.Токены

- Access и refresh токены Яндекс **шифруются** перед сохранением
- Ключ шифрования хранится в **.env** (ENCRYPTION_KEY)
- Автоматическое обновление токенов через refresh_token

JWT

- Access token: 7 дней
- Refresh token: 30 дней
- Хранение сессий в БД
- Возможность завершить все сессии

Зависимости

Основные

- **express** - Web framework
- **better-sqlite3** - Встроенная SQLite база данных
- **jsonwebtoken** - JWT аутентификация
- **bcryptjs** - Хэширование паролей
- **crypto-js** - Шифрование (AES, RSA)
- **ws** - WebSocket сервер
- **multer** - Загрузка файлов
- **winston** - Логирование
- **express-rate-limit** - Ограничение запросов

Для Яндекс.интеграций

- `node-fetch` - HTTP запросы к Яндекс API
- Встроенный `crypto` - Шифрование токенов



Тестирование

```
npm test
```



Разработка

Структура БД

База данных создаётся автоматически при первом запуске. Файл БД: `data/database.db`

Таблицы:

- `users` - Пользователи
- `chats` - Чаты
- `messages` - Сообщения
- `attachments` - Вложения
- `invitations` - Приглашения
- `contacts` - Контакты
- `notifications` - Уведомления
- `sessions` - Сессии
- `devices` - Устройства
- `reports` - Отчёты
- `versions` - Версии приложений

WebSocket

Подключение: `ws://localhost:3001/ws?token=<jwt_token>`

События смотрите в `TODO.md`.



Деплой

Production

1. Установить `NODE_ENV=production`
2. Сгенерировать безопасные секреты для `.env`
3. Настроить Yandex OAuth callback URL
4. Запустить: `npm start`

Мониторинг

Логи сохраняются в:

- `logs/error.log` - Ошибки
- `logs/combined.log` - Все логи

Лицензия

MIT